

# Internship Report

Topic – Infrastructure as Code & Configuration Management

Submitted By:

W.K.D.De.S Gunasekara

DevOps Intern

Davine Technologies

Week 06

Submission Date : /0904/2026

# Introduction & Objectives

## Introduction

This report presents the practical work completed during Week 06 of the DevOps Internship, focusing on Infrastructure as Code (IaC) and Configuration Management.

Terraform was used to practice infrastructure provisioning concepts, including providers, resources, variables, outputs, state management, and the Terraform workflow. Ansible was used for configuration management by creating an inventory and playbook to install Apache, create an application user, manage a service, and create the required project directory.

The practical work demonstrates how Terraform and Ansible can be used together in a DevOps workflow to automate infrastructure and server configuration.

## Objectives

The main objectives of Week 06 were to:

- Understand Infrastructure as Code.
- Learn Terraform fundamentals.
- Practice terraform init, plan, apply, and destroy.
- Understand Terraform state, variables, outputs, and modules.
- Learn Ansible inventory and playbooks.
- Automate package installation, user creation, and service management.
- Understand how Terraform and Ansible work together.

## Infrastructure as Code (IaC)

Infrastructure as Code (IaC) is the practice of defining and managing infrastructure using configuration files instead of performing all infrastructure setup manually. IaC makes infrastructure easier to reproduce, automate, maintain, and version-control.

In this practical work, Terraform was used to demonstrate Infrastructure as Code. The required resource was defined using Terraform configuration files and managed through Terraform commands.

benefits such as:

- Automation
- Repeatability
- Consistency
- Version control
- Reduced manual configuration

## Terraform

Terraform is an Infrastructure as Code tool used to define and manage infrastructure through configuration files. Terraform uses providers to communicate with different platforms and services.

In this practical exercise, the HashiCorp Local Provider was configured. A `local_file` resource was used to create and manage a `welcome.txt` file.

```
resource "local_file" "welcome" {  
  
  filename = "welcome.txt"  
  
  content = var.message  
  
}
```

```
PS C:\Users\USER\DevOps-Internship> cd terraform  
PS C:\Users\USER\DevOps-Internship\terraform> notepad main.tf  
PS C:\Users\USER\DevOps-Internship\terraform> dir
```

Directory: C:\Users\USER\DevOps-Internship\terraform

| Mode   | LastWriteTime      | Length | Name    |
|--------|--------------------|--------|---------|
| ----   | -----              | -----  | ----    |
| -a---- | 8/31/2026 11:52 PM | 282    | main.tf |

```
PS C:\Users\USER\DevOps-Internship\terraform> |
```

# Terraform Workflow

## terraform init

Initializes the Terraform working directory and downloads the required providers.

```
PS C:\Users\USER\DevOps-Internship\terraform> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/local versions matching "~> 2.5"...
- Installing hashicorp/local v2.9.0...
- Installed hashicorp/local v2.9.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\USER\DevOps-Internship\terraform> |
```

## terraform plan

Displays the changes Terraform intends to make before applying them.

```
PS C:\Users\USER\DevOps-Internship\terraform> terraform plan
local_file.welcome: Refreshing state... [id=b5dc2bf9ff10d45ccbc9a09ba9c93b43bd93e]

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
  ~ destroy and then create replacement

Terraform will perform the following actions:

# local_file.welcome must be replaced
/+ resource "local_file" "welcome" {
  ~ content           = "Hello from Terraform! Week 06 DevOps Internship." -> "Welcome to Terraform Variables!" # forces replacement
  ~ content_base64sha256 = "hs7WetTied9XhWgKEZ+lkFo5zEp5Ke9ovAXhH0zEtc=" -> (known after apply)
  ~ content_base64sha512 = "W8GwyZe20LlheiBor93m33FRG+Mb+nqZ001Wki7E3pkQ77A3N+xaPuSuXmuuUXVCnMU1bsD1thZr7QFFy+XA%w==" -> (known after apply)
  ~ content_md5         = "6fb9745215657e1c4a48690f494a08b" -> (known after apply)
  ~ content_sha1        = "b5dc2bf9ff10d45ccbc9a09ba9c93b43bd93e" -> (known after apply)
  ~ content_sha256      = "86ced678d4e279df57856a8628467e9641684b312948a7bda2f017847d3312d7" -> (known after apply)
  ~ content_sha512      = "58110c997b6d008a17a26e8af0de62777d11be31bfa7a99d34d5692244269290efb02337ec5a3ee4ae5e6bae5175429cc5356ec8f5b6166bed0145cbe5c833" -> (known after apply)
  ~ id                 = "b5dc2bf9ff10d45ccbc9a09ba9c93b43bd93e" -> (known after apply)
  # (3 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't
guarantee to take exactly these actions if you run "terraform apply" now.
```

## terraform apply

Applies the Terraform configuration and creates or modifies the required resources.

```
PS C:\Users\USER\DevOps-Internship\terraform> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions
are indicated with the following symbols:
  + create

Terraform will perform the following actions:

# local_file.welcome will be created
+ resource "local_file" "welcome" {
  + content          = "Hello from Terraform! Week 06 DevOps Internship."
  + content_base64sha256 = (known after apply)
  + content_base64sha512 = (known after apply)
  + content_md5       = (known after apply)
  + content_sha1      = (known after apply)
  + content_sha256    = (known after apply)
  + content_sha512    = (known after apply)
  + directory_permission = "0777"
  + file_permission   = "0777"
  + filename         = "welcome.txt"
  + id               = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.welcome: Creating...
local_file.welcome: Creation complete after 0s [id=b5dc2bfafe9ff10d45ccbc9a09ba9c93b43bd93e]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

## terraform destroy

Removes resources managed by the Terraform configuration.

```
PS C:\Users\USER\DevOps-Internship\terraform> terraform destroy
local_file.welcome: Refreshing state... [id=c01bbe3f2172f33689f747cf3267bf3843dcc09]

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
  - destroy

Terraform will perform the following actions:

# local_file.welcome will be destroyed
- resource "local_file" "welcome" {
  content          = "Welcome to Terraform Variables!" -> null
  content_base64sha256 = "C3p48QV1baadJncFFHwzD9Ker2pYcQ8WkX232x/0qg-" -> null
  content_base64sha512 = "bua657z2581dAU83xknJ28EkyfP72w/81X/NSU2dry84di9fidr0acd2ggltzbpCSi45nktvSDGckckjn3mhdnge=" -> null
  content_md5       = "e31484388c74e49351f9af565e493151" -> null
  content_sha1      = "c01bbe3f2172f33689f747cf3267bf3843dcc09" -> null
  content_sha256    = "0b9c96310635da0e6d670ba1451ef9b3f57cbb129c9070cf0d2bcd6f5f7748" -> null
  content_sha512    = "6ee592259cd9491d5d594501df12a78d9044932b056195bfff355ff5b951976bc91e1dbd7e276bd1a71dce080dcba424b3e399d7b6f4831a47248e7de685d9a" -> null
  directory_permission = "0777" -> null
  file_permission   = "0777" -> null
  filename         = "welcome.txt" -> null
  id               = "c01bbe3f2172f33689f747cf3267bf3843dcc09" -> null
}

Plan: 0 to add, 0 to change, 1 to destroy.

Changes to Outputs:
  - file_message = "Welcome to Terraform Variables!" -> null
  - file_name    = "welcome.txt" -> null

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

local_file.welcome: Destroying... [id=c01bbe3f2172f33689f747cf3267bf3843dcc09]
local_file.welcome: Destruction complete after 0s

Destroy complete! Resources: 1 destroyed.
PS C:\Users\USER\DevOps-Internship\terraform> dir

Directory: C:\Users\USER\DevOps-Internship\terraform

Mode                LastWriteTime         Length Name
----                -
d-----          8/31/2026 11:59 PM             .terraform
-a-----          8/31/2026 11:59 PM          1257 .terraform.lock.hcl
-a-----          9/2/2026 18:57 PM           243 main.tf
-a-----          9/2/2026 11:19 PM           115 outputs.tf
-a-----          9/2/2026 11:21 PM           181 terraform.tfstate
```

## Terraform State

Terraform state stores information about resources managed by Terraform. It allows Terraform to track the relationship between the configuration and the actual managed resources.

During this practical exercise, Terraform state was inspected using `terraform show`.

```
PS C:\Users\USER\DevOps-Internship\terraform> terraform show
# local_file.welcome:
resource "local_file" "welcome" {
  content      = "Hello from Terraform! Week 06 DevOps Internship."
  content_base64sha256 = "hs7WeNTied9XhwqGKEZ+1kFoSzEpSKe9ovAXhH0zEtc="
  content_base64sha512 = "WBGWyzZe20Liheibor93mJ3FRG+Mb+nqZ001WkiJEJpKQ77A3N+xaPuSuXmuuUXVCrMU1bsD1thZr7QFFy+XAMw=="
  content_md5      = "6fb9745215657e1cba448690f494a08b"
  content_sha1     = "b5dc2bfaf9ff10d45ccbc9a09ba9c93b43bd93e"
  content_sha256   = "86ced678d4e279df57856a8628467e9641684b312948a7bda2f017847d3312d7"
  content_sha512   = "5811b0c997b6d0b8a17a26e8afdde62777d11be31bfa7a99d34d5692244269290efb02337ec5a3ee4ae5e6bae5175429cc5356ec0f5b6166bed0145cbe5c033"
  directory_permission = "0777"
  file_permission    = "0777"
  filename           = "welcome.txt"
  id                 = "b5dc2bfaf9ff10d45ccbc9a09ba9c93b43bd93e"
}
```

## Terraform Variables and Outputs

### Variables

Terraform variables allow configurable values to be separated from the main resource configuration, making Terraform code easier to reuse and maintain.

```
variable "message" {
  description = "Message for welcome file"

  type      = string

  default   = "Welcome to Terraform Variables!"
}
```

### Outputs

Terraform outputs display useful information after Terraform has processed the configuration.

```
output "file_name" {
  value = local_file.welcome.filename
}
```

```
output "file_message" {
```

```
    value = var.message
}
```

```
PS C:\Users\USER\DevOps-Internship\terraform> New-Item outputs.tf -ItemType File

Directory: C:\Users\USER\DevOps-Internship\terraform

Mode                LastWriteTime         Length Name
----                -
-a----           9/2/2026  11:18 PM              0 outputs.tf

PS C:\Users\USER\DevOps-Internship\terraform> notepad outputs.tf
PS C:\Users\USER\DevOps-Internship\terraform> terraform apply
local_file.welcome: Refreshing state... [id=c01bbbe3f2172f33689f747cf3267bf3843dcc69]

Changes to Outputs:
  + file_message = "Welcome to Terraform Variables!"
  + file_name    = "welcome.txt"

You can apply this plan to save these new output values to the Terraform state,
without changing any real infrastructure.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value: yes

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

file_message = "Welcome to Terraform Variables!"
file_name    = "welcome.txt"
PS C:\Users\USER\DevOps-Internship\terraform> terraform output
file_message = "Welcome to Terraform Variables!"
file_name    = "welcome.txt"
```

## Terraform Modules

A Terraform module is a reusable collection of Terraform configuration files. Modules allow infrastructure configurations to be organized into reusable components.

Modules are useful for reducing duplicated code and creating standardized infrastructure configurations. This practical exercise focused on understanding the concept of Terraform modules rather than implementing a separate custom module.

## Ansible

Ansible is an automation and configuration management tool used to configure servers and automate administrative tasks.

In this practical exercise, Ansible was installed inside Ubuntu running through WSL 2. Ansible was then used to install Apache, create an application user, manage the Apache service, and create a project directory.

```
Processing triggers for man-db (2.13.1-1build1) ...
dilhara@DESKTOP-93QC9B9:/mnt/c/Users/USER/DevOps-Internship/terraform$ ansible --version
ansible [core 2.20.1]
  config file = None
  configured module search path = ['/home/dilhara/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/dilhara/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.14.4 (main, Jun 18 2026, 14:25:02) [GCC 15.2.0] (/usr/bin/python3)
  jinja version = 3.1.6
  pyyaml version = 6.0.3 (with libyaml v0.2.5)
```

## Ansible Inventory

An Ansible inventory defines the hosts that Ansible manages. For this local practical environment, localhost was configured as the managed host.

```
[local]
```

```
localhost ansible_connection=local
```

```
dilhara@DESKTOP-93QC9B9:~/ansible-project$ pwd
/home/dilhara/ansible-project
dilhara@DESKTOP-93QC9B9:~/ansible-project$ nano inventory.ini
dilhara@DESKTOP-93QC9B9:~/ansible-project$ cat inventory.ini
[local]
localhost ansible_connection=local
```

## Ansible Playbook

An Ansible playbook is a YAML file containing automation tasks that should be executed on managed hosts.

The playbook created for this exercise performed the following configuration tasks:

- Installed Apache.
- Created the appuser application user.
- Started and enabled the Apache service.

- Created /home/appuser/devops-project.
- Set the required ownership and permissions.

```
dilhara@DESKTOP-93QC9B9:~/ansible-project$ cat playbook.yml
---
- name: Configure Local Ubuntu Server
  hosts: local
  become: yes

  tasks:
    - name: Install Apache
      apt:
        name: apache2
        state: present
        update_cache: yes

    - name: Create application user
      user:
        name: appuser
        shell: /bin/bash

    - name: Ensure Apache service is running
      service:
        name: apache2
        state: started
        enabled: yes
```

```
dilhara@DESKTOP-93QC9B9:~/ansible-project$ sudo ansible-playbook -i inventory.ini playbook.yml
PLAY [Configure Local Ubuntu Server] *****

TASK [Gathering Facts] *****
[WARNING]: Host 'localhost' is using the discovered Python interpreter at '/usr/bin/python3.14', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [localhost]

TASK [Install Apache] *****
ok: [localhost]

TASK [Create application user] *****
changed: [localhost]

TASK [Ensure Apache service is running] *****
ok: [localhost]

PLAY RECAP *****
localhost : ok=4 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```

## Configuration Change and Idempotence

A configuration change was made through the Ansible playbook by adding a task to create the /home/appuser/devops-project directory.

After applying the change, the directory was successfully created with appuser ownership.

The same playbook was then executed again. The second execution returned changed=0, demonstrating that the desired configuration was already present and Ansible did not make unnecessary changes.

```
dilhara@DESKTOP-93QC989:~/ansible-project$ sudo ansible-playbook -i inventory.ini playbook.yml

PLAY [Configure Local Ubuntu Server] *****

TASK [Gathering Facts] *****
[WARNING]: Host 'localhost' is using the discovered Python interpreter at '/usr/bin/python3.14', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [localhost]

TASK [Install Apache] *****
ok: [localhost]

TASK [Create application user] *****
ok: [localhost]

TASK [Ensure Apache service is running] *****
ok: [localhost]

TASK [Create devops project directory] *****
changed: [localhost]

PLAY RECAP *****
localhost : ok=5 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0

dilhara@DESKTOP-93QC989:~/ansible-project$ sudo ls -ld /home/appuser/devops-project
drwxr-xr-x 2 appuser appuser 4096 Sep  3 23:00 /home/appuser/devops-project
```

```
dilhara@DESKTOP-93QC989:~/ansible-project$ sudo ansible-playbook -i inventory.ini playbook.yml

PLAY [Configure Local Ubuntu Server] *****

TASK [Gathering Facts] *****
[WARNING]: Host 'localhost' is using the discovered Python interpreter at '/usr/bin/python3.14', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [localhost]

TASK [Install Apache] *****
ok: [localhost]

TASK [Create application user] *****
ok: [localhost]

TASK [Ensure Apache service is running] *****
ok: [localhost]

TASK [Create devops project directory] *****
ok: [localhost]

PLAY RECAP *****
localhost : ok=5 changed=0 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```

## Configured Server Verification

After executing the Ansible playbook, the server configuration was verified. Apache was installed successfully, the appuser user existed, and the required project directory was present with the expected ownership and permissions.

```
dilhara@DESKTOP-93QC989:~/ansible-project$ apache2 -v
Server version: Apache/2.4.66 (Ubuntu)
Server built: 2026-07-06T15:33:20
dilhara@DESKTOP-93QC989:~/ansible-project$ id appuser
uid=1005(appuser) gid=1005(appuser) groups=1005(appuser)
dilhara@DESKTOP-93QC989:~/ansible-project$ sudo ls -ld /home/appuser/devops-project
drwxr-xr-x 2 appuser appuser 4096 Sep  3 23:00 /home/appuser/devops-project
dilhara@DESKTOP-93QC989:~/ansible-project$
```

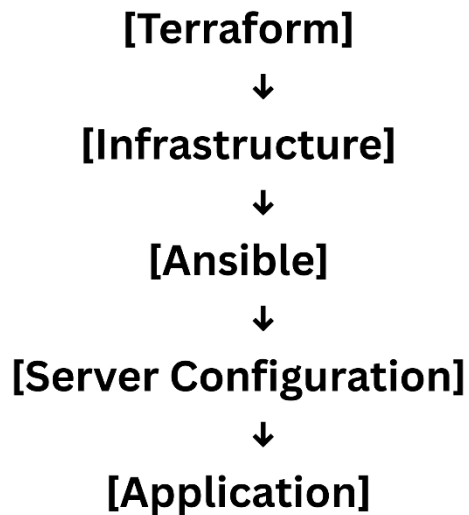
## Terraform vs Ansible

| Feature                         | Terraform                                        | Ansible                                   |
|---------------------------------|--------------------------------------------------|-------------------------------------------|
| <b>Main purpose</b>             | Infrastructure provisioning                      | Configuration management                  |
| <b>Configuration</b>            | .tf files                                        | YAML playbooks                            |
| <b>Infrastructure lifecycle</b> | Create/update/destroy resources                  | Configure existing systems                |
| <b>State</b>                    | Uses Terraform state                             | Does not use Terraform-style state        |
| <b>Week 6 use</b>               | Provider, resource, variables, outputs and state | Packages, users, services and directories |

Terraform and Ansible solve different but complementary problems. Terraform is primarily used to define and provision infrastructure, while Ansible is primarily used to configure systems after infrastructure is available.

Therefore, they can be combined in a DevOps automation workflow.

## Terraform + Ansible Workflow



Terraform was used to define and provision infrastructure using Infrastructure as Code (IaC). The Terraform workflow included terraform init, terraform plan, terraform apply, and terraform destroy.

Ansible was then used for configuration management. Using an inventory and playbook, Ansible installed Apache, created the appuser user, started the Apache service, and created the required project directory.

Running the playbook again resulted in changed=0, demonstrating that the desired configuration was already applied consistently without manual configuration.

## Conclusion

This Week 06 practical provided hands-on experience with Infrastructure as Code and Configuration Management. Terraform was used to understand infrastructure definition, providers, variables, outputs, state, and the infrastructure lifecycle. Ansible was used to automate server configuration through inventories and playbooks.

The exercise demonstrated how infrastructure provisioning and configuration management can be automated and how Terraform and Ansible can complement each other in a DevOps workflow.